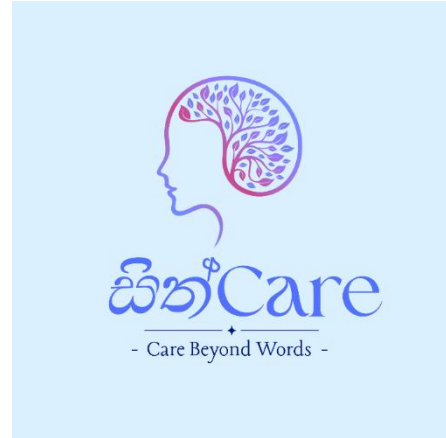




# **SENG 31242 – System Design Project**

## **Final Design Report Draft**

### **Group 10**

**SE/2022/046 - Lochithya Hettiarachchi**

**SE/2022/005 - Anushka Dilinuwan**

**SE/2022/010 - Adhithya Jayawardhana**

**SE/2022/025 - Vindya Wickramathilaka**

**SE/2022/041 - Ramesh Krishan**

**Supervisor – Eng. Dr. Tiroshan Madushanka**

**21/06/2026**

## Declaration

We, the undersigned members of Group 10, hereby declare that this Final Design Report for SithCare - Mental Health Channeling Application is our own original work prepared in partial fulfilment of the requirements for SENG 31242 - System Design Project. All sources of information and references have been duly acknowledged. This document has not been submitted elsewhere for any academic or professional qualification.

| Reg. No.    | Name                    | Signature | Date |
|-------------|-------------------------|-----------|------|
| SE/2022/046 | Lochithya Hettiarachchi |           |      |
| SE/2022/005 | Anushka Dilinuwan       |           |      |
| SE/2022/010 | Adhithya Jayawardhana   |           |      |
| SE/2022/025 | Vindya Wickramathilaka  |           |      |
| SE/2022/041 | Ramesh Krishan          |           |      |

Supervisor: Eng. Dr. Tiroshan Madushanka

Signature: \_\_\_\_\_ Date: \_\_\_\_\_

## **Acknowledgements**

The authors would like to express their sincere gratitude to Eng. Dr. Tiroshan Madushanka, our project supervisor, for his invaluable guidance, constructive feedback, and continuous support throughout the system design phase of this project. We also extend our appreciation to the Software Engineering Teaching Unit, University of Kelaniya, for providing the academic framework and resources necessary for this undertaking.

Our heartfelt thanks go to the mental health professionals, patients, and community stakeholders who participated in our interviews and surveys, providing the essential insights that shaped the requirements and design of SithCare. We also acknowledge the contributions of open-source communities and the developers of the MERN stack technologies that form the foundation of our technical architecture.

## Abstract

SithCare ("Care Beyond Words") is a web-based mental health channelling application designed specifically for the Sri Lankan context to address the critical gap in accessible, private, and stigma-free mental health support. The system enables patients to discover qualified mental health specialists, book online or in-person appointments, access telemedicine consultations, and receive guidance through an AI-powered triage chatbot named SithBot. This Final Design Report consolidates the complete system analysis and design artefacts produced during the SENG 31242 System Design Project. The report encompasses a comprehensive Software Requirements Specification (SRS) derived from structured interviews, questionnaires, and competitive analysis; and a detailed Software Design Specification (SDS) covering the Layered MVC architecture, MERN technology stack, class diagrams, sequence diagrams, database design, user interface wireframes, and security design. The system supports distinct role-based dashboards for patients, clinic administrators, platform administrators, volunteer listeners, and community moderators, with multi-clinic management, a live patient queue tracker, anonymous community support, and an emergency response subsystem. All design decisions are traceable to the non-functional requirements through an explicit mapping framework.

**Keywords:** *mental health, telemedicine, appointment booking, system design, SRS, SDS, Sri Lanka, MERN stack, SithCare, SENG 31242*

# Table of Contents

|                                           |           |
|-------------------------------------------|-----------|
| <b>Declaration .....</b>                  | <b>2</b>  |
| <b>Acknowledgements .....</b>             | <b>3</b>  |
| <b>Abstract .....</b>                     | <b>4</b>  |
| <b>Table of Contents .....</b>            | <b>5</b>  |
| <b>List of Figures .....</b>              | <b>8</b>  |
| <b>List of Tables .....</b>               | <b>10</b> |
| <b>Abbreviations .....</b>                | <b>12</b> |
| <b>1. Introduction .....</b>              | <b>14</b> |
| 1.1 Background.....                       | 14        |
| 1.2 Problem Statement.....                | 14        |
| 1.3 Aims and Objectives .....             | 15        |
| 1.4 Scope .....                           | 16        |
| 1.4.1 In Scope .....                      | 16        |
| 1.4.2 Out of Scope .....                  | 16        |
| 1.5 Organisation of the Report .....      | 17        |
| <b>2. System Analysis.....</b>            | <b>18</b> |
| 2.1 Fact-Finding Techniques .....         | 18        |
| 2.2 Existing System Analysis .....        | 18        |
| 2.2.1 Current State and Pain Points ..... | 18        |
| 2.3 Use Case Analysis .....               | 19        |
| 2.3.1 System Actors.....                  | 19        |
| 2.3.2 Use Case List.....                  | 19        |
| 2.3.3 Activity Diagrams.....              | 20        |
| 2.4 Functional Requirements.....          | 21        |
| 2.5 Non-Functional Requirements.....      | 21        |
| 2.5.1 Performance Requirements .....      | 21        |

|                                                       |           |
|-------------------------------------------------------|-----------|
| 2.5.2 Security Requirements.....                      | 22        |
| 2.6 Alternative Solutions and Feasibility.....        | 22        |
| 2.6.1 Alternatives Considered.....                    | 22        |
| 2.6.2 Feasibility Assessment.....                     | 22        |
| 2.6.3 MoSCoW Scope Prioritisation .....               | 23        |
| <b>3. System Design .....</b>                         | <b>24</b> |
| 3.1 Architectural Design.....                         | 24        |
| 3.1.1 Architectural Pattern .....                     | 24        |
| 3.1.2 Quality Attribute Justification .....           | 24        |
| 3.1.3 Component Design .....                          | 25        |
| 3.1.4 Deployment Architecture.....                    | 25        |
| 3.2 Technology Stack.....                             | 26        |
| 3.3 Detailed Design .....                             | 27        |
| 3.3.1 Class Diagram .....                             | 27        |
| 3.3.2 Sequence Diagrams .....                         | 27        |
| 3.3.3 State Transition Diagrams .....                 | 27        |
| 3.3.4 Entity-Relationship Diagram.....                | 28        |
| 3.4 User Interface Design .....                       | 28        |
| 3.5 Security Design .....                             | 28        |
| 3.5.1 Authentication and Authorization.....           | 28        |
| 3.5.2 Data Protection .....                           | 28        |
| 3.5.3 OWASP Top 10 Considerations.....                | 28        |
| 3.6 Non-Functional Design Decisions.....              | 29        |
| <b>4. Conclusion .....</b>                            | <b>31</b> |
| 4.1 Objectives Met.....                               | 31        |
| 4.2 Limitations.....                                  | 31        |
| 4.3 Future Work.....                                  | 32        |
| <b>References.....</b>                                | <b>33</b> |
| <b>Appendix A: Risk and Mitigation Register .....</b> | <b>35</b> |

|                                                      |           |
|------------------------------------------------------|-----------|
| <b>Appendix B: Glossary of Terms .....</b>           | <b>36</b> |
| <b>Appendix C: GitHub Repository Artefacts .....</b> | <b>37</b> |

## List of Figures

1. Use Case Diagram - Register and Login (UC-01)
2. Use Case Diagram - Manage User Profile and Consent (UC-02)
3. Use Case Diagram - Search Mental Health Specialist (UC-03)
4. Use Case Diagram - Book Appointment (UC-04)
5. Use Case Diagram - Manage Doctor Availability (UC-05)
6. Use Case Diagram - Approve / Reschedule Appointment (UC-06)
7. Use Case Diagram - Join Telemedicine Consultation (UC-07)
8. Use Case Diagram - Process Payment and Generate Receipt (UC-08)
9. Use Case Diagram - Track Live Patient Queue (UC-09)
10. Use Case Diagram - Request Emergency Support (UC-10)
11. Use Case Diagram - Use Anonymous Community Consultation (UC-11)
12. Use Case Diagram - Complete AI Triage Questionnaire (SithBot) (UC-012)
13. Activity Diagram - User Authentication (AD-01)
14. Activity Diagram - Appointment Booking (AD-02)
15. Activity Diagram - Emergency Support (AD-03)
16. Activity Diagram - Telemedicine Consultation (AD-04)
17. Activity Diagram - Community Posting (AD-05)
18. Component Diagram
19. Deployment Diagram
20. Class Diagram
21. Sequence Diagram - Register and Login (UC-01)
22. Sequence Diagram - Manage User Profile and Consent (UC-02)
23. Sequence Diagram - Search Mental Health Specialist (UC-03)
24. Sequence Diagram - Book Appointment (UC-04)
25. Sequence Diagram - Manage Doctor Availability (UC-05)
26. Sequence Diagram - Approve / Reschedule Appointment (UC-06)
27. Sequence Diagram - Join Telemedicine Consultation (UC-07)
28. Sequence Diagram - Process Payment and Generate Receipt (UC-08)
29. Sequence Diagram - Track Live Patient Queue (UC-09)
30. Sequence Diagram - Request Emergency Support (UC-10)



31. Sequence Diagram - Use Anonymous Community Consultation (UC-11)
32. Sequence Diagram - Complete AI Triage Questionnaire (SithBot) (UC-012)
33. State Transition Diagram - Register and Login (UC-01)
34. State Transition Diagram - Manage User Profile and Consent (UC-02)
35. State Transition Diagram - Search Mental Health Specialist (UC-03)
36. State Transition Diagram - Book Appointment (UC-04)
37. State Transition Diagram - Manage Doctor Availability (UC-05)
38. State Transition Diagram - Approve / Reschedule Appointment (UC-06)
39. State Transition Diagram - Join Telemedicine Consultation (UC-07)
40. State Transition Diagram - Process Payment and Generate Receipt (UC-08)
41. State Transition Diagram - Track Live Patient Queue (UC-09)
42. State Transition Diagram - Request Emergency Support (UC-10)
43. State Transition Diagram - Use Anonymous Community Consultation (UC-11)
44. State Transition Diagram - Complete AI Triage Questionnaire (SithBot) (UC-012)
45. Entity-Relationship Diagram
46. Patient Dashboard Wireframes
47. Clinic Admin Dashboard Wireframes
48. SithCare Admin Dashboard Wireframes

## List of Tables

1. User Classes and Characteristics
2. Operating Environment
3. Assumptions and Dependencies
4. Fact-Finding Techniques Used
5. Competitive Analysis Summary
6. System Boundary and Actors
7. Use Case List
8. Use Case Descriptions for all 12 use cases
9. Activity diagram descriptions for main 5 use cases
10. Sequence Diagram Descriptions for all 12 cases
11. Functional Requirements – Authentication and Access Control
12. Functional Requirements - Profile and Consent Management
13. Functional Requirements - Specialist Search and Discovery
14. Functional Requirements - Appointment Management
15. Functional Requirements – Telemedicine
16. Functional Requirements – Queue Management and Clinic Operations
17. Functional Requirements – Payment Processing
18. Functional Requirements – Emergency Support
19. Functional Requirements – AI Chatbot (SithBot)
20. Functional Requirements – Community Features
21. Functional Requirements – Localization and Communication
22. Non-Functional Requirements - Performance
23. Non-Functional Requirements – Security
24. Non-Functional Requirements - Usability
25. Non-Functional Requirements - Reliability
26. Non-Functional Requirements – Scalability
27. Non-Functional Requirements - Maintainability
28. Feasibility Assessment
29. MoSCoW Scope Prioritisation
30. Appendix A – Risk and Mitigation Register

31. Appendix B – Glossary of Sri Lankan Mental Health Terms
32. Appendix C – Planned Repository Artefacts
33. External Interface Specifications
34. Internal Module Interfaces
35. API Endpoint Groups
36. RBAC Role Permissions
37. OWASP Top 10 Considerations
38. Non-Functional Design Decision - Performance
39. Non-Functional Design Decision – Security
40. Non-Functional Design Decision – Usability
41. Non-Functional Design Decision – Reliability
42. Non-Functional Design Decision – Scalability
43. Non-Functional Design Decision – Maintainability

## Abbreviations

| Abbreviation  | Definition                                     |
|---------------|------------------------------------------------|
| <b>SRS</b>    | Software Requirements Specification            |
| <b>SDS</b>    | Software Design Specification                  |
| <b>FR</b>     | Functional Requirement                         |
| <b>NFR</b>    | Non-Functional Requirement                     |
| <b>UC</b>     | Use Case                                       |
| <b>RBAC</b>   | Role-Based Access Control                      |
| <b>JWT</b>    | JSON Web Token                                 |
| <b>OTP</b>    | One-Time Password                              |
| <b>MFA</b>    | Multi-Factor Authentication                    |
| <b>LKR</b>    | Sri Lankan Rupee                               |
| <b>API</b>    | Application Programming Interface              |
| <b>TLS</b>    | Transport Layer Security                       |
| <b>AES</b>    | Advanced Encryption Standard                   |
| <b>MVP</b>    | Minimum Viable Product                         |
| <b>MoSCoW</b> | Must-have, Should-have, Could-have, Won't-have |
| <b>PDPA</b>   | Personal Data Protection Act                   |
| <b>SLMC</b>   | Sri Lanka Medical Council                      |
| <b>SLPA</b>   | Sri Lanka Psychological Association            |
| <b>CBT</b>    | Cognitive Behavioural Therapy                  |
| <b>MERN</b>   | MongoDB, Express.js, React.js, Node.js         |
| <b>ODM</b>    | Object Data Modeling                           |
| <b>SPA</b>    | Single Page Application                        |
| <b>REST</b>   | Representational State Transfer                |
| <b>CDN</b>    | Content Delivery Network                       |
| <b>CI/CD</b>  | Continuous Integration / Continuous Deployment |

| Abbreviation | Definition                            |
|--------------|---------------------------------------|
| WCAG         | Web Content Accessibility Guidelines  |
| OWASP        | Open Web Application Security Project |
| SQL          | Structured Query Language             |
| NoSQL        | Not Only SQL                          |
| EMU          | English Metric Units                  |

---

# 1. Introduction

## 1.1 Background

Mental health care in Sri Lanka faces significant challenges that limit access to professional support for millions of citizens. According to the WHO Mental Health Atlas 2024, Sri Lanka has only 2.5 mental health workers per 100,000 population, indicating a severe shortage of qualified professionals. The National Mental Health Policy 2020-2030 emphasises community support, early detection, continuing care, and stigma reduction as national priorities. Despite these policy directives, the treatment gap remains substantial, with many individuals who suffer from mental disorders not seeking professional help due to barriers including stigma, poor awareness, transport limitations, language barriers, and financial constraints.

The current landscape of mental health service delivery relies primarily on outpatient department queues at government hospitals and manual telephone bookings at private psychiatric clinics. These traditional methods create significant pain points: the fear of being overheard during phone bookings, lack of digital queue tracking resulting in waiting times of one to four hours at clinic premises, absence of telemedicine options for rural patients, and limited knowledge of available crisis support helplines. The Sri Lankan Digital Health Blueprint 2023 supports the adoption of digital tools to improve healthcare accessibility and timely service delivery, creating a favourable policy environment for technology-enabled mental health solutions.

SithCare ("Care Beyond Words") is proposed as a comprehensive web-based mental health channelling application specifically designed for the Sri Lankan context. The platform aims to bridge the gap between patients and mental health professionals by providing a secure, private, and stigma-free digital environment for discovering specialists, booking appointments, accessing telemedicine consultations, and receiving community support.

## 1.2 Problem Statement

The mental health care ecosystem in Sri Lanka is characterised by several interconnected problems that collectively limit access to professional support:

- ❑ No centralised directory of licensed mental health professionals is publicly accessible, making it difficult for patients to find verified specialists.
- ❑ Telephone-only booking creates stigma, as patients fear being overheard by family members or colleagues when making appointments.
- ❑ No digital queue tracking exists; patients wait at clinic premises for extended periods without information about estimated wait times.
- ❑ Payment records are maintained in paper ledgers, with no digital receipts or payment history available to patients.
- ❑ No telemedicine option exists for rural patients or those with mobility issues, effectively excluding a significant portion of the population.
- ❑ Crisis support helplines are unknown to most patients, and numbers are not widely publicised or easily accessible during emergencies.

SithCare addresses these problems through a unified digital platform that combines specialist discovery, appointment management, telemedicine, emergency support, and anonymous community features tailored specifically for the Sri Lankan context.

### **1.3 Aims and Objectives**

The primary aim of this project is to design a scalable, secure, and user-friendly mental health channelling platform that improves access to mental health services in Sri Lanka. The specific objectives are:

1. To conduct comprehensive requirements engineering through structured interviews, surveys, and competitive analysis to identify user needs and system requirements.
2. To produce a professional-grade Software Requirements Specification (SRS) documenting all functional and non-functional requirements.
3. To design a scalable software architecture and produce a detailed Software Design Specification (SDS) including class diagrams, sequence diagrams, database design, and user interface wireframes.
4. To plan and manage the project using GitHub Projects, adhering to industry-standard agile practices.

5. To produce professional documentation artefacts including the final design report, meeting notes, and architectural decision records.

## 1.4 Scope

The scope of the SithCare platform for the design phase (SENG 31242) and subsequent development phase (SENG 34213) encompasses the following:

### 1.4.1 In Scope

- ☐ Patient registration and authentication with OTP verification and role-based access control.
- ☐ Mental health specialist search and discovery with filtering by category, language, location, and availability.
- ☐ Appointment booking, rescheduling, and cancellation with SMS and email notifications.
- ☐ Online payment processing through PayHere gateway in Sri Lankan Rupees (LKR).
- ☐ Telemedicine video consultations with one-tap phone fallback.
- ☐ Live patient queue tracking with proximity notifications.
- ☐ Emergency support subsystem with helpline display and consent-based location sharing.
- ☐ Anonymous community consultation forum with pseudonymous identities.
- ☐ AI-powered triage chatbot (SithBot) for specialist category recommendations.
- ☐ Multi-language support (Sinhala, Tamil, English) and Adult Mode for elderly users.

### 1.4.2 Out of Scope

- ☐ Native mobile applications for Android and iOS (future consideration).
- ☐ Integration with government electronic health record (EHR) systems.
- ☐ Insurance claim processing and complex fundraising automation.
- ☐ AI clinical diagnosis (SithBot provides triage guidance only, not diagnoses).
- ☐ Automatic emergency service dispatch without explicit user confirmation.



## **1.5 Organisation of the Report**

The remainder of this report is organised as follows. Chapter 2 presents the System Analysis, encompassing fact-finding techniques, existing system analysis, use case modelling, functional and non-functional requirements, and feasibility assessment. Chapter 3 details the System Design, including architectural decisions, technology stack selection, detailed design artefacts, user interface design, security design, and non-functional design decision mapping. Chapter 4 provides the Conclusion, summarising objectives met, identifying limitations, and outlining future work. References and Appendices follow the main chapters.

## 2. System Analysis

### 2.1 Fact-Finding Techniques

The requirements for SithCare were elicited using multiple formal fact-finding methods as mandated by the SENG 31242 course guidelines. The techniques employed and their key outcomes are summarised below:

**Technique: Structured Interviews:** Details: 5 interviews with mental health patients | Key Findings: Stigma is primary barrier; patients prefer anonymous search; elderly need simplified UX

**Online Questionnaire:** 150 respondents via Google Forms | Quantitative data on service gaps, preferred features, willingness to use digital platforms

**Specialist Survey:** 6 interviews with mental health specialists | Identified pain points and requirements for digital professional activities

**Document Analysis:** WHO Atlas 2024, MoH Policy 2020-2030, Digital Health Blueprint 2023 | 2.5 workers per 100,000; significant treatment gap; government digital health push

**Competitive Analysis:** BetterHelp, Talkspace, Wysa, NalandaCare | No existing platform provides full Sri Lankan localisation with multi-clinic management

### 2.2 Existing System Analysis

#### 2.2.1 Current State and Pain Points

Mental health care access in Sri Lanka currently relies on outpatient department queues at government hospitals, manual telephone bookings at private psychiatric clinics, and informal WhatsApp groups for community support. The competitive analysis revealed significant gaps in existing solutions. No existing platform combines full Sri Lankan localisation (Sinhala/Tamil/English, LKR currency, PayHere payments), multi-clinic management, live queue tracking, AI triage, emergency support, anonymous community features, and volunteer listener support in a single integrated solution.

## 2.3 Use Case Analysis

The use case analysis identified twelve primary use cases spanning patient engagement, clinical operations, administrative functions, and community support. The system boundary encompasses interactions between six primary actor categories and four external system actors.

### 2.3.1 System Actors

1. **Patient:** Registers, searches specialists, books appointments, joins consultations, uses emergency/community features
2. **Elderly Patient / Adult Mode User:** Uses simplified interface for essential functions: booking, guidance, and emergency access
3. **Mental Health Specialist / Doctor:** Manages profile, availability, appointments, telemedicine, session notes
4. **Clinic Administrator:** Manages clinic branches, schedules, queues, payments, appointment approvals
5. **Platform Administrator:** Manages users, roles, clinics, content, reports, compliance
6. **Volunteer Listener:** Provides supportive listening sessions following code of conduct
7. **Community Moderator:** Reviews reported content, volunteer sessions, recovery stories
8. **AI Chatbot (SithBot):** Automated triage interacts with patients to suggest specialist categories
9. **Payment Gateway (PayHere):** Processes online payments and returns transaction status
10. **Video Consultation Service:** Hosts and manages telemedicine video rooms
11. **SMS/Email Notification Service:** Delivers appointment reminders, OTPs, confirmations

### 2.3.2 Use Case List

Twelve use cases were identified and fully documented with descriptions, activity diagrams, and sequence diagrams:

1. **UC-01 - Register and Login:** Account creation with OTP, secure login, role-based routing.

2. **UC-02 - Manage User Profile and Consent:** Profile updates, privacy consent, emergency contacts.
3. **UC-03 - Search Mental Health Specialist:** Filter by category, language, location, availability.
4. **UC-04 - Book Appointment:** Slot selection, payment, confirmation, notifications.
5. **UC-05 - Manage Doctor Availability:** Define, update, block available time slots.
6. **UC-06 - Approve/Reschedule Appointment:** Review, approve, reject, reschedule requests.
7. **UC-07 - Join Telemedicine Consultation:** Secure video room with phone fallback.
8. **UC-08 - Process Payment and Generate Receipt:** PayHere integration, cash tracking, PDF receipts.
9. **UC-09 - Track Live Patient Queue:** Real-time token display, proximity notifications.
10. **UC-10 - Request Emergency Support:** Helpline display, crisis contact, location sharing.
11. **UC-11 - Use Anonymous Community:** Pseudonymous posts, responses, moderation.
12. **UC-12 - Complete AI Triage (SithBot):** Guided questions, specialist recommendation.

A separate document has been attached including all the Use-cases and descriptions to the GitHub Repository.

GitHub: [Source Link](#)

### 2.3.3 Activity Diagrams

Five activity diagrams were produced: (AD-01) User Authentication covering role-based login and registration with OTP; (AD-02) Appointment Booking encompassing specialist search, slot selection, payment flow, and confirmation; (AD-03) Emergency Support detailing button activation, disclaimer display, consent confirmation, and helpline fallback; (AD-04) Telemedicine Consultation modelling video room link generation, join flow, and phone fallback; and (AD-05) Community Posting depicting anonymous post creation, pseudonymous identity masking, and moderation rules. All diagrams are committed to the repository in source (.drawio) and exported (SVG) formats.

Activity Diagrams for the major use Use-cases have been uploaded to GitHub Repository.

GitHub: [Source Link](#)

## 2.4 Functional Requirements

Functional requirements are specified with unique identifiers (FR-XX), MoSCoW priority, source reference, and verification method. Fifty-two functional requirements were identified across eleven categories. Key categories include:

- ❑ Authentication and Access Control (6 requirements): Registration with OTP, password complexity, RBAC, password reset, auto logout, MFA
- ❑ Appointment Management (6 requirements): Booking, cancellation, approval workflow, emergency same-day, reminders, availability management
- ❑ Payment Processing (4 requirements): PayHere integration, cash payment tracking, PDF receipt generation, refund processing
- ❑ Emergency Support (5 requirements): Persistent emergency button, consent-based location sharing, helpline display, non-substitution disclaimer, crisis event logging
- ❑ AI Chatbot - SithBot (3 requirements): Triage without login, crisis keyword detection, non-diagnosis disclaimer
- ❑ Community Features (2 requirements): Anonymous posting with pseudonymous IDs, content moderation tools
- ❑ Localisation and Communication (4 requirements): Sinhala/Tamil/English support, LKR currency, local emergency numbers, multilingual notifications

## 2.5 Non-Functional Requirements

### 2.5.1 Performance Requirements

1. **NFR-P-01:** Page load  $\leq 3$  seconds on 4G for 95% of requests
2. **NFR-P-02:** Booking transaction  $\leq 5$  seconds (excluding PayHere redirect)
3. **NFR-P-03:** Support minimum 500 concurrent users
4. **NFR-P-04:** Queue update propagation within 60 seconds
5. **NFR-P-05:** SithBot response  $\leq 2$  seconds for 90% of queries

### 2.5.2 Security Requirements

1. **NFR-S-01:** TLS 1.3 for all data in transit
2. **NFR-S-02:** AES-256 for patient data at rest
3. **NFR-S-03:** JWT access: 15 minutes, refresh: 7 days
4. **NFR-S-04:** OWASP Top 10 mitigations
5. **NFR-S-05:** Health records restricted to authoring doctor
6. **NFR-S-06:** Pseudonymous community IDs
7. **NFR-S-07:** No card data stored; PayHere handles PCI-DSS
8. **NFR-S-08:** Auth events logged with timestamp and IP

## 2.6 Alternative Solutions and Feasibility

### 2.6.1 Alternatives Considered

- ☐ Alternative A - Manual Phone/Walk-in Booking: Simple but causes delays, no transparency, no digital records, significant stigma barriers.
- ☐ Alternative B - Open-Source Platform (OpenMRS): Primarily for hospital EMR; lacks patient-facing UX, community features, requires substantial customisation.
- ☐ Alternative C - Healthcare SaaS (Medplum): Optimised for US market; Sri Lankan requirements need significant custom development with vendor dependency.
- ☐ Alternative D - Social Media/WhatsApp Groups: Poor privacy, misinformation, no moderation, no professional connection or appointment management.

### 2.6.2 Feasibility Assessment

- ☐ Technical Feasible: Team skills include React, Node.js, SQL, REST APIs. Jitsi is open-source. PayHere provides sandbox.
- ☐ Economic Feasible: Development cost is team effort only. Free-tier cloud hosting available. PayHere sandbox has no monthly fee.
- ☐ Operational Feasible: Clinic admins capable of web dashboards per interviews. Patient UI designed for low-literacy users.
- ☐ Schedule Feasible with MoSCoW: 8-week design sufficient for MVP. Advanced features deferred to SENG 34213.

- Legal/Ethical Feasible with constraints: Requires PDPA controls, RBAC, data minimisation, consent recording, medical disclaimers.

### 2.6.3 MoSCoW Scope Prioritisation

- Must Have: Authentication, role-based dashboards, specialist search, appointment booking with PayHere/cash, availability management, emergency support, SMS/email notifications, multilingual support.
- Should Have: Telemedicine, PDF receipts, live queue tracker, doctor verification, content moderation, Adult Mode, multi-branch clinic management.
- Could Have: Anonymous community forum, volunteer listener portal, AI chatbot (SithBot), specialist ratings, analytics dashboard.
- Won't Have (v1.0): AI clinical diagnosis, automatic emergency dispatch, government EHR integration, native mobile apps, insurance processing.

A separate SRS (Software Requirements Specifications) document has been created and uploaded to a GitHub and to Google Drive.

GitHub: [GitHub Source Link](#)

Google Drive: [Google Drive Link](#)

## 3. System Design

### 3.1 Architectural Design

#### 3.1.1 Architectural Pattern

SithCare employs a Client-Server Architecture utilising a Layered (N-Tier) model on the backend, tightly integrated with the Model-View-Controller (MVC) design pattern. The application is built using the MERN stack (MongoDB, Express.js, React.js, Node.js). The architectural layers are:

- Presentation Layer (Client): React-based Single Page Application handling UI, routing, and state management for Patients, Specialists, and Administrators.
- Application/Controller Layer (Server): Express.js REST APIs that intercept HTTP requests, validate payloads, and manage Role-Based Access Control via JSON Web Tokens.
- Business Logic Layer (Server): Node.js services managing core functionalities such as appointment scheduling, queue management, AI triage (SithBot), and emergency routing.
- Data Access Layer (Database): Mongoose Object Data Modeling (ODM) wrapping a NoSQL MongoDB database.

A separate Architecture Specification document has been created and uploaded to a GitHub and to Google Drive.

GitHub: [GitHub Source](#)

#### 3.1.2 Quality Attribute Justification

The chosen architecture directly supports the Non-Functional Requirements:

- Scalability: Node.js non-blocking I/O supports 500 concurrent users and 5x traffic spikes. MongoDB flexible schema accommodates schema-per-tenant design for multiple clinic branches.



- **Maintainability:** MVC and Layered approach enforce separation of concerns. JavaScript across the entire stack reduces context switching and improves code maintainability.
- **Security:** Dedicated Controller layer verifies JWTs and enforces RBAC before business logic execution. MongoDB field-level encryption aids PDPA compliance.
- **Reliability:** React virtual DOM ensures sub-3-second page loads. Separation of external services ensures third-party failures do not crash the primary application.
- **Usability:** React combined with Tailwind CSS provides a responsive, intuitive experience. Simplified navigation flows reduce cognitive load.\

### 3.1.3 Component Design

The platform consists of six major components: (1) Patient Module handling registration, authentication, specialist search, booking, queue tracking, emergency support, and community interaction; (2) Mental Health Specialist Module managing appointments, schedules, telemedicine, and patient communication; (3) Clinic Administration Module providing clinic profile management, doctor allocation, queue management, and appointment monitoring; (4) SithCare Admin Module for oversight of all portals, user management, and risk control; (5) Community Support Module enabling anonymous consultation, recovery stories, peer support, and volunteer listener sessions; and (6) AI Assistance Module (SithBot) providing symptom-based triage, specialist recommendations, and navigation assistance.

### 3.1.4 Deployment Architecture

The deployment topology utilises free-tier cloud services optimised for MERN stack applications. Client devices access the SPA via HTTPS (TLS 1.3). Static React assets are hosted on Vercel or Netlify with global CDN. The backend Node.js Express API runs on Render or Railway with SSL termination. MongoDB Atlas Free Tier provides cloud-hosted database persistence. All external integrations communicate securely over HTTPS.

### 3.2 Technology Stack

The platform is built using the MERN stack, selected for its unified JavaScript ecosystem, high development velocity, and alignment with team skills.

- Frontend: React + Vite (component-based architecture, fast rendering, reusable UI, scalable SPA)
- Styling: Tailwind CSS (rapid UI development, utility-first responsive design, consistent design system)
- Backend: Node.js + Express.js (lightweight, event-driven async architecture, large middleware ecosystem)
- Database: MongoDB Atlas (flexible document schema, cloud-native, high scalability)
- ODM: Mongoose (schema validation, query handling, middleware, model abstraction)
- Auth: JWT + bcrypt (stateless authentication, secure password storage, RBAC support)
- Payments: PayHere Gateway (local Sri Lankan payment support, easy MERN integration)
- SMS: Twilio SMS API (reliable delivery, appointment reminders, emergency notifications)
- Email: Nodemailer (lightweight integration, OTP verification, password recovery)
- Video: Jitsi Meet API (open-source, lightweight, minimal infrastructure overhead)
- Maps: Google Maps API (accurate clinic mapping, consent-based emergency location sharing)
- Hosting: Vercel/Netlify + Render (free-tier support, automatic SSL, seamless GitHub integration)

A separate Architecture Technology Stack document has been created and uploaded to a GitHub and to Google Drive.

GitHub: [GitHub Source](#)

## 3.3 Detailed Design

### 3.3.1 Class Diagram

The class diagram defines the complete object model including all classes with full attribute and method specifications. Key entity classes include User (with subclasses Patient, Doctor, ClinicAdmin, PlatformAdmin), Appointment, Clinic, Specialization, AvailabilitySlot, Payment, QueueEntry, CommunityPost, CrisisEvent, and ConsultationSession. Service classes include AuthService, AppointmentService, QueueManager, PaymentService, NotificationService, TelemedicineService, EmergencyHandler, CommunityService, and SithBotService.

A separate Class Diagram document has been created and uploaded to a GitHub and to Google Drive.

GitHub: [GitHub Source](#)

### 3.3.2 Sequence Diagrams

Twelve sequence diagrams were produced, one per use case, modelling interactions between actors, frontend components, backend controllers, services, and external systems. Key interactions include OTP verification (UC-01), PayHere webhook handling (UC-08), Jitsi video room lifecycle (UC-07), and emergency fallback (UC-10).

A separate Sequence Diagrams document has been created and uploaded to a GitHub and to Google Drive.

GitHub: [GitHub Source](#)

### 3.3.3 State Transition Diagrams

State diagrams for all twelve use cases model entity lifecycles. Notable state machines include Appointment (Pending Approval to Confirmed to In Progress to Completed/Cancelled), Queue Entry (Waiting to Called to Completed/No-show), and Payment (Pending to Processing to Paid/Failed to Refunded).

### 3.3.4 Entity-Relationship Diagram

The ER diagram models the MongoDB document-based schema with collections for Users, Clinics, Doctors, Appointments, Payments, QueueEntries, CommunityPosts, CrisisEvents, and ConsultationSessions. Relationships use reference fields and embedded documents with indexing for high-frequency query paths.

## 3.4 User Interface Design

The UI is organised into four dashboard interfaces. The Patient Dashboard features a calming design with teal accents. Key screens include landing page with SithBot access, specialist search with map integration, booking wizard, video consultation waiting room, live queue tracker, community forum, and emergency activation. Adult Mode provides simplified navigation with 18px minimum font. The Clinic Admin Dashboard provides appointment lists, availability calendars, queue control, payment tracking, and reports. The SithCare Admin Dashboard offers user management, clinic verification, crisis log review, and system analytics.

## 3.5 Security Design

### 3.5.1 Authentication and Authorization

Authentication uses JWT access tokens (15-minute expiry) and refresh tokens (7-day expiry). Passwords stored as bcrypt salted hashes. Optional MFA for Clinic and Platform Administrators. RBAC enforces six distinct roles with clearly defined permissions.

### 3.5.2 Data Protection

Data in transit uses HTTPS with TLS 1.3. Patient data at rest encrypted with AES-256. Payment card data never stored; PayHere handles PCI-DSS. Consent records stored with timestamp for audit.

### 3.5.3 OWASP Top 10 Considerations

- ☐ Broken Access Control: RBAC middleware, ownership checks, route protection, admin logging
- ☐ Cryptographic Failures: TLS 1.3, AES-256, bcrypt, no card data storage

- ☐ Injection: Backend validation, Mongoose schema validation, sanitised filters
- ☐ Insecure Design: Controller validation, service isolation, audit logging
- ☐ Security Misconfiguration: Environment variables, secure CORS, disabled debug output
- ☐ Vulnerable Components: Dependency review, package updates, security alerts
- ☐ Auth Failures: OTP verification, strong passwords, token expiry, MFA, account lockout
- ☐ Data Integrity: Pull requests, code review, CI checks, webhook verification
- ☐ Logging Failures: Auth events, role changes, payments, emergencies logged

### 3.6 Non-Functional Design Decisions

The following mapping shows how each NFR is addressed by design decisions:

1. **NFR-P-01 (Page load  $\leq 3s$ ):** React+Vite, Vercel/Netlify CDN, compressed images, Tailwind CSS
2. **NFR-P-02 (Booking  $\leq 5s$ ):** Dedicated Appointment Service, simple DB operations, async notifications
3. **NFR-P-03 (500 concurrent users):** Node.js non-blocking I/O, stateless REST APIs, horizontal scaling
4. **NFR-S-01 (TLS 1.3 in transit):** HTTPS for all frontend-backend and external API communication
5. **NFR-S-02 (AES-256 at rest):** MongoDB Atlas storage encryption for sensitive data
6. **NFR-U-01 (SUS score  $\geq 70$ ):** Simple role-based dashboards, clear navigation, guided workflows
7. **NFR-U-03 (Adult Mode 18px font):** Simplified navigation, larger text, reduced feature set
8. **NFR-R-01 (99.5% uptime):** Cloud hosting with uptime monitoring, critical error logging
9. **NFR-R-04 (Emergency if non-critical fails):** Static helpline configuration, independent emergency UI
10. **NFR-SC-01 (Horizontal scaling):** Stateless REST APIs, JWT session state

- 11. NFR-SC-02 (200 clinics no migration):** MongoDB flexible schema, clinic-based tenant separation
- 12. NFR-M-01 (Consistent coding standards):** GitHub, PRs, code reviews, linting, CONTRIBUTING.md
- 13. NFR-M-02 (70% unit test coverage):** Modular backend services for independent testing

A separate SDS (Software Design Document) document has been created and uploaded to a GitHub and to Google Drive.

GitHub: [GitHub Source Link](#)

Google Drive: [Google Drive Link](#)

## 4. Conclusion

### 4.1 Objectives Met

The SithCare project has successfully achieved all five stated objectives. Comprehensive requirements engineering was conducted through structured interviews with five mental health patients, an online questionnaire with 150 respondents, six specialist interviews, and document analysis of WHO Mental Health Atlas 2024, National Mental Health Policy 2020-2030, and Digital Health Blueprint 2023.

A professional-grade Software Requirements Specification was produced, documenting twelve use cases with complete descriptions, fifty-two functional requirements with MoSCoW prioritisation, twenty non-functional requirements, and a feasibility assessment with alternative solution analysis. All requirements are traceable to their source and include verification methods.

A detailed Software Design Specification was developed specifying a Layered MVC architecture with MERN stack implementation, complete class diagram, twelve sequence diagrams, five activity diagrams, twelve state transition diagrams, entity-relationship diagram, UI wireframes for all four dashboard interfaces, and security design addressing all OWASP Top 10 risk areas.

The project was managed using GitHub Projects with four sprint milestones, epic labels, structured issue backlog, and story point estimation. All artefacts committed to the GitHub organisation repository with version control, branch protection, and conventional commit messages.

This Final Design Report consolidates all analysis and design artefacts into a single document conforming to the SENG 31242 report specification, providing a complete handoff package for the SENG 34213 development phase.

### 4.2 Limitations

- Design assumes free-tier cloud services with limitations on bandwidth, storage, and concurrent connections.

- ☐ Telemedicine quality depends on Jitsi SDK performance and network conditions in rural Sri Lanka.
- ☐ AI chatbot (SithBot) provides triage guidance only and cannot provide clinical diagnoses.
- ☐ Emergency support requires active internet; static helplines serve as fallback only.
- ☐ Payment processing limited to PayHere gateway; international methods not supported initially.
- ☐ No government EHR integration, which may limit continuity of care across providers.

### **4.3 Future Work**

- ☐ Native Android and iOS mobile applications for improved accessibility.
- ☐ Advanced AI capabilities including NLP for SithBot and sentiment analysis for content moderation.
- ☐ Mood tracking and progress monitoring for longitudinal mental health management.
- ☐ Comprehensive analytics dashboard with predictive insights for resource allocation.
- ☐ Integration with government EHR systems through Health Information Exchange standards.
- ☐ Insurance claim processing and fundraising automation.
- ☐ Usability testing with target user groups to validate SUS score and refine Adult Mode.
- ☐ Performance testing under realistic Sri Lankan network conditions.



## References

Health Information Unit (2023). Mental Health Service Providers Directory 2023. Ministry of Health, Sri Lanka. ( <https://www.health.gov.lk/wp-content/uploads/2025/04/Mental-Health-Service-Providers-Directory-2023.pdf> )

NCBI Bookshelf (2024). Mental health prevalence, healthcare use and access between 2018 and 2022 in Sri Lanka: an analysis of survey data. (<https://www.ncbi.nlm.nih.gov/books/NBK610790/> )

Ministry of Health (2020). National Mental Health Policy 2020-2030. Government of Sri Lanka. ( <https://www.health.gov.lk/wp-content/uploads/2022/10/National-Mental-Health-Policy-2020-2030-compressed.pdf> )

Ministry of Health (2023). Sri Lanka Digital Health Blueprint. Health Information Unit. (<https://www.health.gov.lk/wp-content/uploads/2023/11/Digital-Health-Blue-Print-Full-Book-01.11.2023-Final.pdf> )

World Health Organization (2024). Mental Health Atlas 2024. WHO Press, Geneva. ([https://cdn.who.int/media/docs/default-source/mental-health/mental-health-atlas-2024-country-profiles/sri-lanka.pdf?sfvrsn=8202fd50\\_3](https://cdn.who.int/media/docs/default-source/mental-health/mental-health-atlas-2024-country-profiles/sri-lanka.pdf?sfvrsn=8202fd50_3) )

University of Kelaniya (2026). SENG 31242 System Design Project: Course Guideline and Industrial Standards. Software Engineering Teaching Unit.

IEEE (1998). IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications. IEEE. ( <https://standards.ieee.org/standard/830-1998.html> )

IEEE (2009). IEEE Std 1016-2009: Standard for Information Technology - Systems Design - Software Design Descriptions. IEEE. ( <https://standards.ieee.org/standard/1016-2009.html> )

MongoDB Inc. (2024). MongoDB Documentation. ( <https://www.mongodb.com/docs/> )

Meta Platforms (2024). React Documentation. ( <https://react.dev/> )

OpenJS Foundation (2024). Node.js Documentation. ( <https://nodejs.org/docs/latest/api/> )

PayHere (2024). PayHere API Documentation. ( <https://support.payhere.lk/api-&-mobile-sdk/> )

Jitsi Meet API Documentation. ( <https://jitsi.github.io/handbook/docs/dev-guide/dev-guide-iframe/> )

OWASP Foundation (2024). OWASP Top 10:2021. ( <https://owasp.org/Top10/> )

World Wide Web Consortium (W3C), *Web Content Accessibility Guidelines (WCAG) 2.1*. ( <https://www.w3.org/TR/WCAG21/> )

## Appendix A: Risk and Mitigation Register

- Video consultation latency: Low likelihood, Medium impact. Mitigation: Keep telemedicine optional; SE Asia servers; phone fallback.
- AI chatbot medical liability: Low likelihood, High impact. Mitigation: Triage only; prohibit diagnosis; mandatory disclaimer; professional review.
- Scope too large for timeline: Medium likelihood, High impact. Mitigation: MoSCoW prioritisation; focus MVP; GitHub Projects tracking.
- Patient data privacy breach: Medium likelihood, High impact. Mitigation: AES-256 at rest; TLS 1.3; pseudonymous IDs; strict RBAC; PDPA compliance.
- Crisis feature misuse: Low likelihood, Very High impact. Mitigation: Explicit confirmation; per-session consent; non-substitution disclaimer; static fallback helplines.
- Volunteer listener burnout: Medium likelihood, Medium impact. Mitigation: Mandatory training; code of conduct; 60-min limits; report button; moderator review.
- PayHere integration failure: Low likelihood, High impact. Mitigation: Cash payment alternative; slot reservation; webhook retry; status notification.

## Appendix B: Glossary of Terms

- SLMC: Sri Lanka Medical Council. Statutory body for registering medical practitioners.
- SLPA: Sri Lanka Psychological Association. Professional body for psychologists.
- Sumithrayo: Volunteer-based Sri Lankan suicide prevention hotline. Helpline: 1926.
- CCCline: Community Counselling Centre helpline. Helpline: 0112 696 666.
- Sahanaya: Counselling and rehabilitation centre. Helpline: 0115 679 900.
- PDPA: Personal Data Protection Act - proposed Sri Lankan data protection legislation.
- SithBot: AI triage chatbot. Provides specialist category recommendations only.
- Adult Mode: Simplified interface for elderly/low-digital-literacy users.
- Telemedicine: Remote consultation using video/audio communication tools.
- Pseudonymous ID: Platform identifier that hides real identity in community features.

## Appendix C: GitHub Repository Artefacts

All project artefacts are maintained in the GitHub organisation repository `seng31242-sithcare-2026`. The repository structure follows the SENG 31242 guidelines:

- ☐ `documents/srs/`: SRS draft and final documents, use case descriptions, requirements tables
- ☐ `documents/sds/`: SDS document, architecture decisions, design diagrams
- ☐ `documents/diagrams/`: Use case, activity, sequence, state, class, ER diagrams (source + exports)
- ☐ `documents/pitch/`: Idea pitch deck and project approval form
- ☐ `documents/presentation/`: Design presentation slides
- ☐ `documents/meetings/`: Meeting notes, stand-up logs, sprint retrospectives
- ☐ `research/`: Interview transcripts, survey analysis, competitive analysis, literature review
- ☐ `prototypes/`: UI wireframes, Figma exports, low-fidelity prototypes